Java Programming Homework | Making Decisions, Looping

For each question, write, compile, and test a Java class. Save each class in a separate .java file using the filename shown.

Question 1: Weekly Parking Fees

Write a Java program that calculates parking fees for several visits. The program first reads an integer n, representing the number of visits. For each visit, read an integer representing the number of parking hours.

Calculate the fee for each visit using these rules:

- The first two hours cost $30 per hour.
- Each additional hour costs $20.
- The maximum fee for one visit is $150.

Use a loop to process all visits and decision statements to calculate each fee. Display the fee for each visit on a separate line. After processing all visits, display the total parking fee.

You may assume that $1 \leq n \leq 20$ and that each visit lasts between 1 and 24 whole hours. Do not use arrays. Your program must read from standard input without displaying input prompts.

Question 2: Student Score Summary

Write a Java program that summarizes a class's examination results. The program first reads an integer n, representing the number of students. It then reads n integer scores, one for each student.

Classify each score using these rules:

- A score of 80 or above is classified as Excellent.
- A score from 60 to 79 is classified as Pass.
- A score below 60 is classified as Fail.

Use a loop to process the scores. Display the classification for each student on a separate line. After processing all scores, display the number of students in each classification and the highest score.

You may assume that $1 \leq n \leq 50$ and that all scores are between 0 and 100. Do not use arrays or sorting methods. Read from standard input without displaying input prompts, and follow the output labels shown below.

Question 3: Bank Account Transactions

Write a Java program that processes transactions for a bank account. The program first reads a nonnegative integer representing the initial balance. It then repeatedly reads an integer transaction amount until it reads 0.

Process transactions using these rules:

- A positive amount represents a deposit and increases the balance.
- A negative amount represents a withdrawal.
- Accept a withdrawal only if the account has sufficient funds.
- If a withdrawal would make the balance negative, reject it and leave the balance unchanged.

After each accepted transaction, display Balance: followed by the updated balance. For each rejected withdrawal, display Rejected. When the program reads 0, stop processing and display the final balance and the number of rejected withdrawals.

The terminating 0 is not a transaction and must not produce a balance update. Use a loop and decision statements. Read from standard input without displaying input prompts.

Question 4: Number Classification Report

Write a Java program that analyzes a sequence of integers. The program first reads a positive integer n, indicating how many numbers will follow. It then reads exactly n integers.

For each number, determine whether it is positive, negative, or zero. Also determine whether it is even or odd. Display both classifications on one line, separated by one space. Use the labels Positive, Negative, Zero, Even, and Odd.

After processing all numbers, display the number of positive values, negative values, and zeros. Also display the sum of all even numbers, including negative even numbers.

Use a loop, decision statements, and the remainder operator %. Remember that zero is an even number. You may assume that $1 \leq n \leq 100$ and that each number is between $-1{,}000$ and 1,000. Do not use arrays. Read from standard input without displaying input prompts.

Question 5: Hollow Number Square

Write a Java program that displays a hollow square of numbers. The program reads an integer n, representing both the number of rows and the number of columns.

Use nested loops to visit every position in the square. Decide what to display using these rules:

- If a position is in the first or last row, display its column number.
- If a position is in the first or last column, display its column number.
- Otherwise, display an asterisk *.

Column numbers begin at 1. Separate adjacent symbols with exactly one space, and do not print spaces at the end of a line.

After displaying the square, print the total number of asterisks on a new line using the label shown below. Count asterisks as they are printed.

You may assume that $3 \leq n \leq 9$. Use loops and decision statements without arrays. Read from standard input without displaying input prompts.